

Syntax Errors

Out of the two types of errors you will encounter, the first and easiest one to solve is the syntax error. These are also called as parsing errors and are caused because of incorrect syntax used in building the code (duh! right?). E.g. if you miss the colon at the end of the line defining the function, you will see a syntax error pointing to the line and giving the line number so you can rectify it. In case the error came from a script, file name and line number are also printed.

Exceptions

The second and more insidious kind of error. Your entire code may be syntactically correct but can still throw up an error when you execute it and are known as exceptions. This causes the emergence of The Hulk. Let's look at a sample code and the exceptions that it will raise

- `10 * (1/0)`
- `4 + spam*3`
- `'2' + 2`

The first line will cause the exception, `ZeroDivisionError: division by zero`. You go back to the code and change it to say, `10 * (1/1)`. The second line will give the error, `NameError: name 'spam' is not defined` and even if you change that, the last line will show, `TypeError: must be str, not int`. A simple code that raises three exceptions- `ZeroDivisionError`, `NameError` and `TypeError`!

These are all part of standard exceptions and are all inbuilt in Python. Here's a small sample the inbuilt errors

- **Exception**- Base class for all exceptions
- **StopIteration**- Raised when the next() method of an iterator does not point to any object.
- **StandardError**- Base class for all built-in exceptions except StopIteration and SystemExit.
- **ArithmeticError**- Base class for all errors that occur for numeric calculation.
- **OverflowError**- Raised when a calculation exceeds maximum limit for a numeric type.
- **FloatingPointError**- Raised when a floating point calculation fails.
- **ZeroDivisionError**- Raised when division or modulo by zero takes place for all numeric types.
- **AttributeError**- Raised in case of failure of attribute reference or assignment.